AUTOBOTs: Autonomous Security Engineer

Design and Architecture Specifications

Date: 19/11/2025

Group Members:

Syed Ahmed Haseeb	**2022557**
Muhammad Bilal	**2022359**
Muhammad Bilal Aslam	**2022361**
Muhammad Hassaan Shah	**2022385**

Supervisor:
Hafiz Syed Ahmed Qasim

Co-Supervisor:
Muhammad Ahmad Nawaz

Faculty of Computer Science and Engineering

GIK INSTITUTE
OF ENGINEERING SCIENCES AND TECHNOLOGY

AUTOBOTs: Autonomous Security Engineer

Revision History:

Revision History	***Date***	***Comments***
1.00	10/11/2025	Initial draft
2.00	19/11/2025	Diagram improvements and finalizing report

Document Approval:

The following document has been accepted and approved by the following:

Signature	***Date***	***Name***
	19/11/2025	Hafiz Syed Ahmed Qasim
	19/11/2025	Muhammad Ahmad Nawaz

List of Contents

1. INTRODUCTION 6

1.1. PURPOSE 6
1.2. PRODUCT SCOPE 6
1.3. OVERVIEW 7

2. THE OVERALL DESCRIPTION 7

2.1. PRODUCT PERSPECTIVE 7

3. WORK BREAKDOWN STRUCTURE 8

3.1. PROJECT INITIALIZATION 8
3.2. REQUIREMENT GATHERING AND ANALYSIS 8
3.3. SYSTEM DESIGN 8
3.4. FEATURE DEVELOPMENT 8
3.5. USER INTERFACE AND USER EXPERIENCE 9
3.6. DOCUMENTATION AND FINALIZATION 9
3.7. PROJECT MANAGEMENT 9

4. DESIGN 10

4.1 ARCHITECTURAL DESIGN 10
4.2. WHY WE CHOOSE TWO-TIER CLIENT-SERVER WITH LAYERED ARCHITECTURE DESIGN? 11
4.3. MODULE IDENTIFICATION 12
1. User Interaction Layer 12
2. AI Orchestration Layer 12
3. Analysis & Tools Layer 12
4. Execution Layer 12
5. Cache Layer 12
6. Reporting Layer 12

5. 4+1 ARCHITECTURE VIEW MODEL 13

5.1. USE CASE VIEW 13
5.2. LOGICAL VIEW: 14
5.3. DEVELOPMENT VIEW 15
5.4. PROCESS VIEW 17
5.5. PHYSICAL VIEW 19
5.6. USER INTERFACE DESIGN 19

List of Figures

Figure 1: Work Breakdown Structure.....9
Figure 2: Architectural Design.....10
Figure 3: Use Case View.....13
Figure 4: Logical View.....14
Figure 5: Components Diagram.....14
Figure 6: Development View.....15
Figure 7: Context Diagram.....16
Figure 8: Process View.....17
Figure 9: Sequence Diagram for a complete pen-testing process on a target.....17
Figure 10: Activity Diagram.....18
Figure 11: Physical View.....19
Figure 12: AUTOBOTs Local Application UI.....19
Figure 13: Integrated with 5IRE.....20
Figure 14: AUTOBOTs integrated with Cursor.....20

List of Tables

Table 1: Terms used in this document and their description…….....06

1. INTRODUCTION

1.1. PURPOSE

The primary purpose of AUTOBOTs is to develop an autonomous cybersecurity framework capable of performing, validating, and scaling penetration testing and vulnerability remediation with minimal manual oversight. The system enhances security testing efficiency through AI agents that respond to evolving threats and complex environments while reducing false positives via reproducible proof-of-concept validation and intelligent vulnerability assessment. By preserving context and continuity across extended security campaigns, AUTOBOTs ensures accurate and comprehensive results. It is designed to scale efficiently across large numbers of targets, streamline compliance and remediation processes, and minimize organizational delays while improving accountability. Ultimately, AUTOBOTs aims to redefine cybersecurity practices by shifting from manual, human-dependent testing to an autonomous, intelligent, and continuously adaptive security pattern.

1.2. PRODUCT SCOPE

AUTOBOTs is an AI-driven penetration testing and vulnerability management system designed to autonomously identify, validate, and remediate security flaws across web applications, code-bases, and digital infrastructures. The system operates through a multi-agent architecture where specialized AI agents collaborate to perform reconnaissance, exploit discovery, proof-of-concept validation, and automated patching with minimal human input. By leveraging Large Language Models (LLMs) and vulnerability intelligence, AUTOBOTs delivers context-aware analysis, adaptive testing strategies, and reproducible results to eliminate false positives and improve accuracy.

The product's scope extends to supporting continuous and scalable testing across large enterprise environments, maintaining global context across complex, multi-step campaigns, and generating structured vulnerability reports with validated findings. It will also include mechanisms for compliance tracking with frameworks such as PCI DSS 4.0, HIPAA, and ISO/IEC 27001:2022, helping organizations meet regulatory and audit requirements efficiently.

Table 1: Terms used in this document and their description

Name	Description
API	Application Programming Interface
LLMs	Large Language Models
MCP Gateway	The security guard & coordinator (the middle manager)
Tool Adapter	Executes tool commands safely in a sandbox
HIPAA	Health Insurance Portability and Accountability Act
PCI DSS	Payment Card Industry Data Security Standard

1.3. OVERVIEW

AUTOBOTs is a revolutionary AI-driven web application penetration testing tool that leverages advanced concepts like Modern Context Protocol, multi-agent architecture powered by Large Language Models (LLMs). Its primary goal is to autonomously perform reconnaissance on target web applications, map their attack surfaces, identify and exploit vulnerabilities to assess their potential impact, recommend mitigation strategies, and deliver comprehensive security reports to organizations.

This system integrates autonomous AI agents that have over various reconnaissance and security tools, enabling it to effectively map an organization's digital infrastructure and identify potential entry points that attackers could exploit. By detecting these vulnerabilities before malicious actors can, AUTOBOTs empowers organizations to proactively safeguard their assets, maintain confidentiality, integrity, and privacy, and implement effective mitigation mechanisms well in advance.

Through this proactive and intelligent approach, organizations can significantly reduce financial losses, prevent data breaches, and minimize post-attack recovery costs. AUTOBOTs not only enhances security visibility but also ensures long-term resilience by embedding continuous, autonomous, and intelligent vulnerability assessment into the organization's security principles.

2. THE OVERALL DESCRIPTION

2.1. PRODUCT PERSPECTIVE

AUTOBOTs is a standalone, web-based AI penetration testing platform designed to integrate seamlessly into an organization's existing security and development infrastructure. It functions as an intelligent offensive security component within the broader cybersecurity ecosystem, complementing tools such as vulnerability scanners and DevOps pipelines.

The system operates through a multi-agent architecture, where each agent performs specific security functions such as reconnaissance, exploitation, validation, and reporting under centralized coordination. AUTOBOTs interacts with target web applications, APIs, and databases using secure communication channels and leverages over various tools to gather and analyze data.

AUTOBOTs is designed to provide actionable insights and validated findings that can be directly utilized by security teams for remediation.

AUTOBOTs receives instructions from users and performs OSINT Gathering, Exploit Development, Vulnerability Scanning and much more. It depends on integrated reconnaissance and exploitation tools, external vulnerability databases, and secure API communication. The system requires stable network connectivity and adequate computational resources to support concurrent testing and LLM processing.

3. WORK BREAKDOWN STRUCTURE

3.1. Project Initialization

In this phase, the project scope is defined, including the goals of creating an autonomous, multi-agent penetration testing system. The development methodology is selected, and team roles are assigned to ensure smooth execution.

3.2. Requirement Gathering and Analysis

This step involves collecting user requirements, identifying essential system capabilities such as AI agent interaction, autonomous execution, real-time adaptation, and advanced reporting. Relevant standards, existing solutions, and system dependencies are analyzed to validate feasibility and ensure AUTOBOTs can outperform current tools.

3.3. System Design

The system architecture is designed, including multi-agent communication flows, LLM-integration logic, secure communication channels, and interface design for IDE/LLM integration. Logical views such as class diagrams, component diagrams, and sequence diagrams are created to guide development.

3.4. Feature Development

Core functionalities such as OSINT gathering, vulnerability scanning, exploit generation, autonomous execution, real-time adaptive decision-making, cloud and web application testing, and reporting modules are developed. AI reasoning, multi-agent orchestration, exploit validation, and data security components are also implemented.

3.5. User Interface and User Experience

Front-end components such as the IDE plugin interface and LLM selection dashboard are implemented. The system is integrated into platforms like VSCode, Cursor, Roo Code, and 5IRE. Interfaces are optimized for usability, workflow clarity, and seamless interaction between users and AI agents.

3.6. Documentation and Finalization

Technical documentation, system architecture diagrams, and user guides (for developers, testers, and security teams) are prepared. Project reports, demonstrations, and presentations are completed for delivery.

3.7. Project Management

Throughout the project, continuous management was ensured for timely delivery, tracking of milestones, communication with stakeholders, and adherence to cybersecurity standards. Regular reviews ensure the system aligns with the intended capabilities and quality benchmarks.

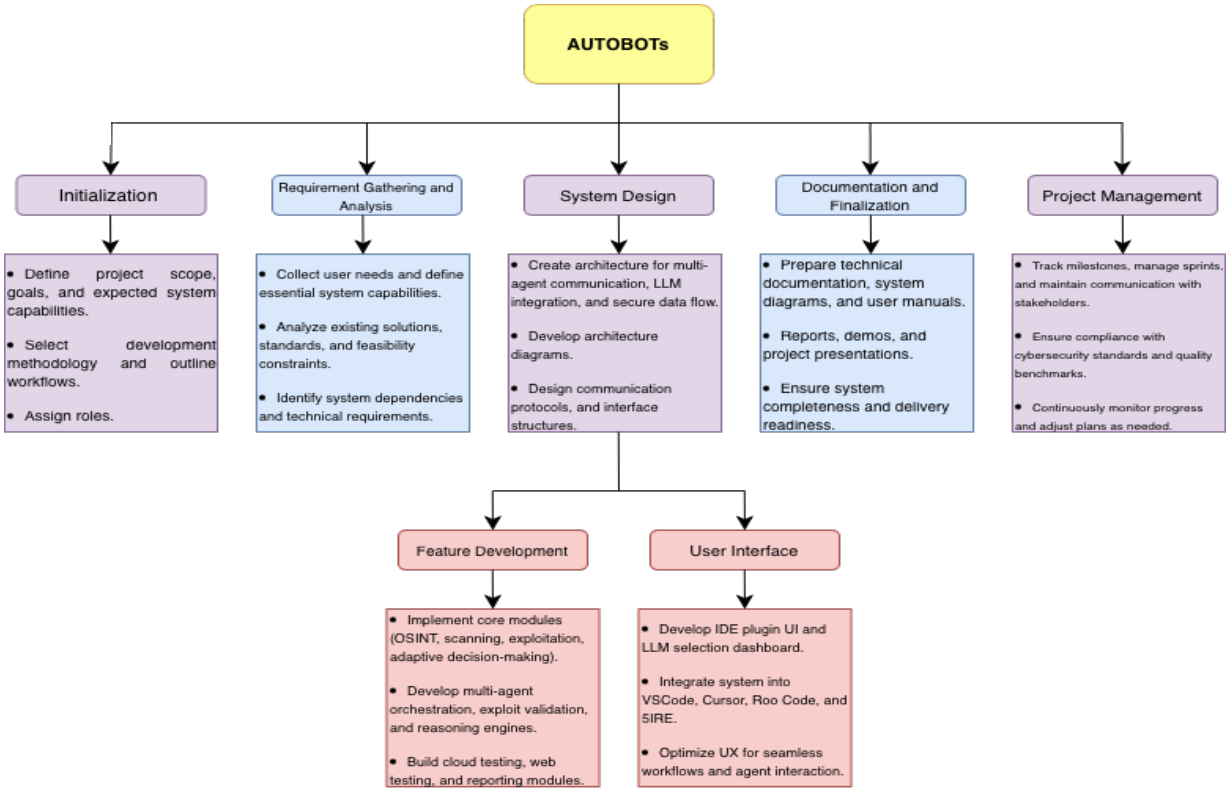

Figure 1: Work Breakdown Structure

4. Design

4.1 ARCHITECTURAL DESIGN

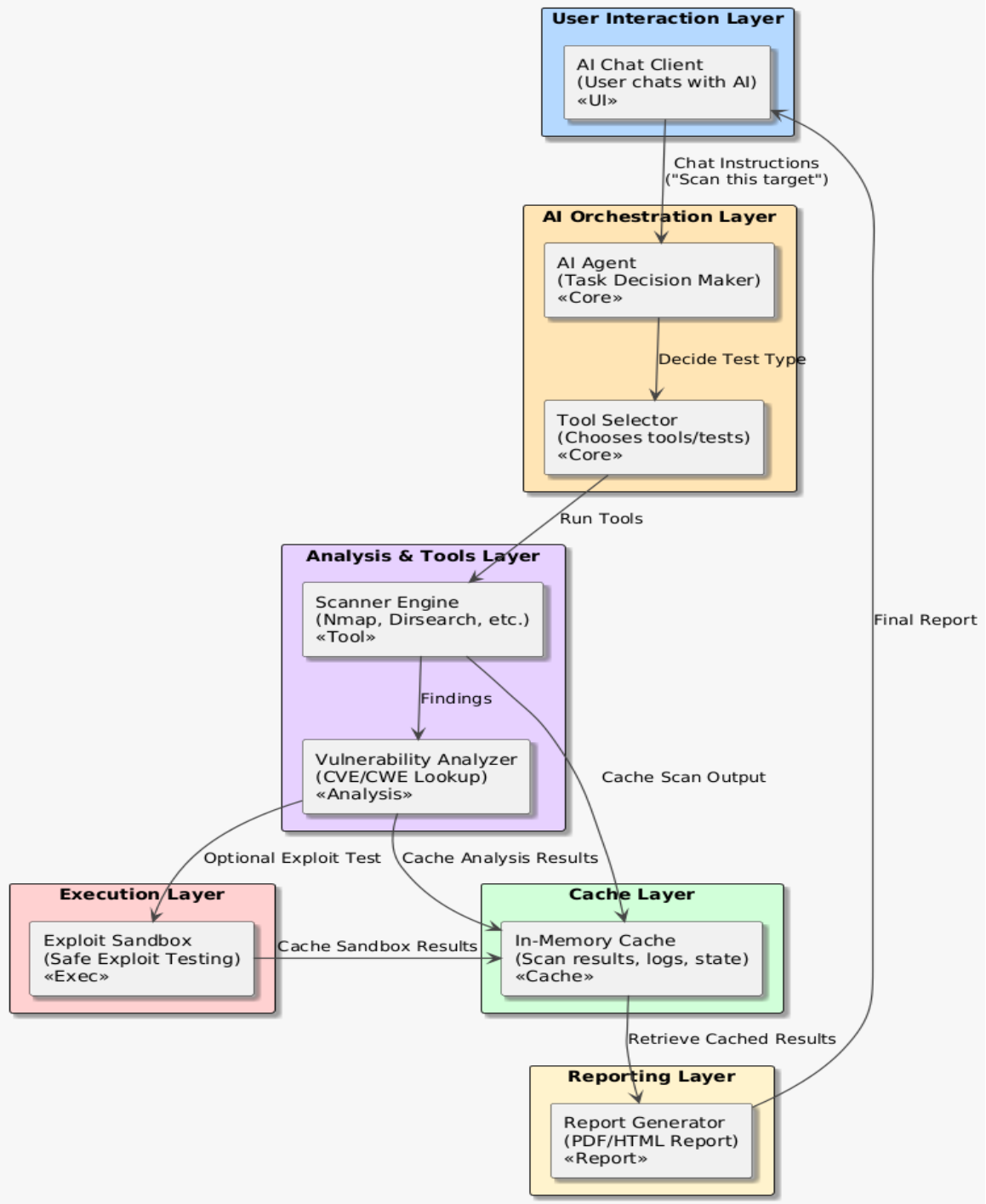

Figure 2: Architectural Design

The AUTOBOTs system follows a Two-Tier Client–Server Architecture combined with a Layered Architectural Model. The client tier consists of the User Interaction Layer, where users interact through IDE plugins or a chat-based AI interface. The server tier hosts all core AUTOBOTs functionalities, divided into multiple layers:

- **AI Orchestration Layer**: Manages AI agent reasoning, tool selection, and decision-making workflows.
- **Analysis & Tools Layer**: Runs scanners, performs vulnerability analysis, and evaluates findings.
- **Execution Layer**: Executes safe sandbox exploits when required.
- **Cache Layer**: Stores temporary scan results, logs, and state for performance optimization.
- **Reporting Layer**: Generates structured reports (PDF, HTML) based on findings and exploit validations.

4.2. Why we choose Two-tier client-server with layered architecture design?

1. **Real-Time Responsiveness:** Users need quick responses inside IDEs or AI chat interfaces. A two-tier model minimizes intermediate layers, reducing latency.

2. **Secure, Controlled Backend Execution:** All sensitive operations, scanning, exploit generation, data analysis, run on the secure server tier, preventing risk on user machines.

3. **Clear Separation of Roles:** The layered backend (Orchestration, Tools, Execution, Cache, Reporting) ensures modular development, easy debugging, and maintainability.

4. **Scalability & Extensibility:** Each layer can scale independently. For example:
    - Cache can expand for large-scale scans
    - Analysis layer can add more scanning engines
    - Execution layer can add advanced sandbox components

5. **Supports Multi-Agent Workflows:** The architecture allows AI agents to communicate, coordinate tasks, and execute workflows in real time without blocking the user side.

6. **Scalability & Extensibility:** Each layer can scale independently. For example:
    - Cache can expand for large-scale scans
    - Analysis layer can add more scanning engines
    - Execution layer can add advanced sandbox components

7. **Supports Multi-Agent Workflows:** The architecture allows AI agents to communicate, coordinate tasks, and execute workflows in real time without blocking the user side.

4.3. MODULE IDENTIFICATION

1. **User Interaction Layer**
    - IDE Plugin UI
    - AI Chat Client
    - Command Input Module

2. **AI Orchestration Layer**
    - AI Agent (Decision Engine)
    - Task Planner
    - Tool Selector Module

3. **Analysis & Tools Layer**
    - Scanner Engine (Nmap, Dirsearch, etc.)
    - Vulnerability Analyzer (CVE/CWE Matching)
    - Result Aggregator

4. **Execution Layer**
    - Exploit Sandbox
    - PoC Execution Manager

5. **Cache Layer**
    - In-Memory Cache
    - Scan Result Storage
    - Temporary Log Storage

6. **Reporting Layer**
    - Report Generator (PDF/HTML)
    - Documentation Formatter
    - Summary/Insights Engine

5. 4+1 ARCHITECTURE VIEW MODEL

5.1. Use Case View

Provides a high-level representation of how different user groups (analysts, developers, administrators) interact with AUTOBOTs to perform autonomous security testing tasks.

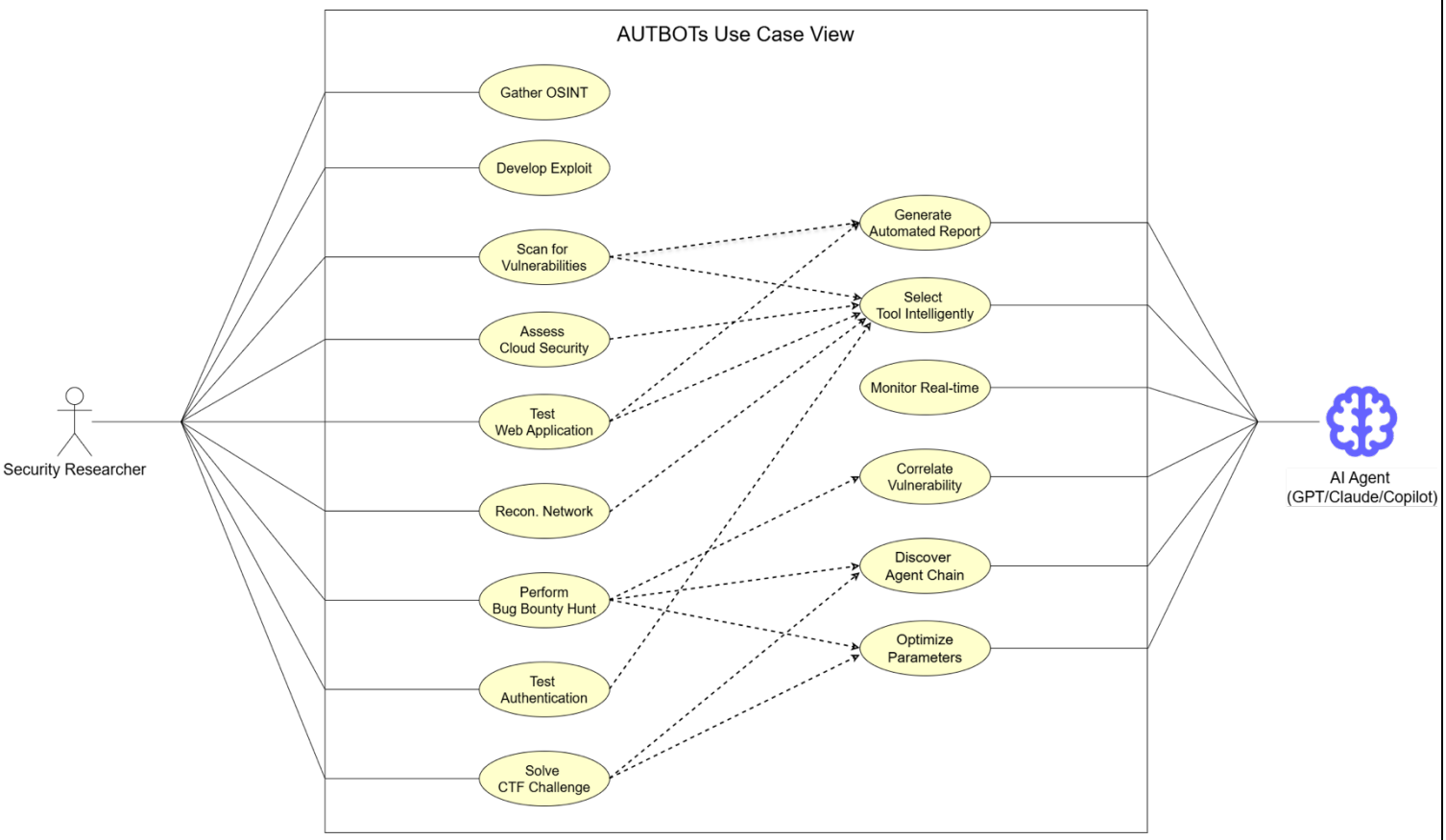

Figure 3: Use Case View

5.2. Logical View:

Defines the system's functional structure, showing how modules like AI agents, scanning engines, orchestration logic, and reporting components collaborate.

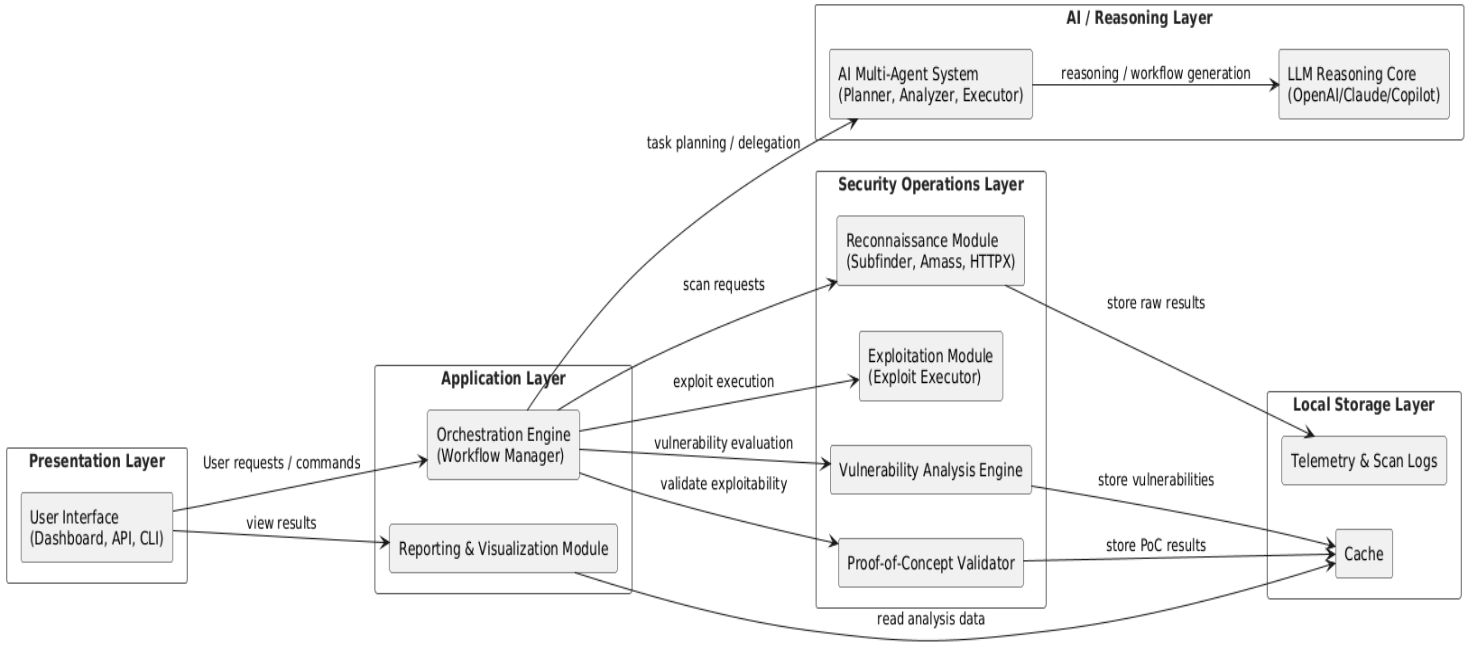

Figure 4: Logical View

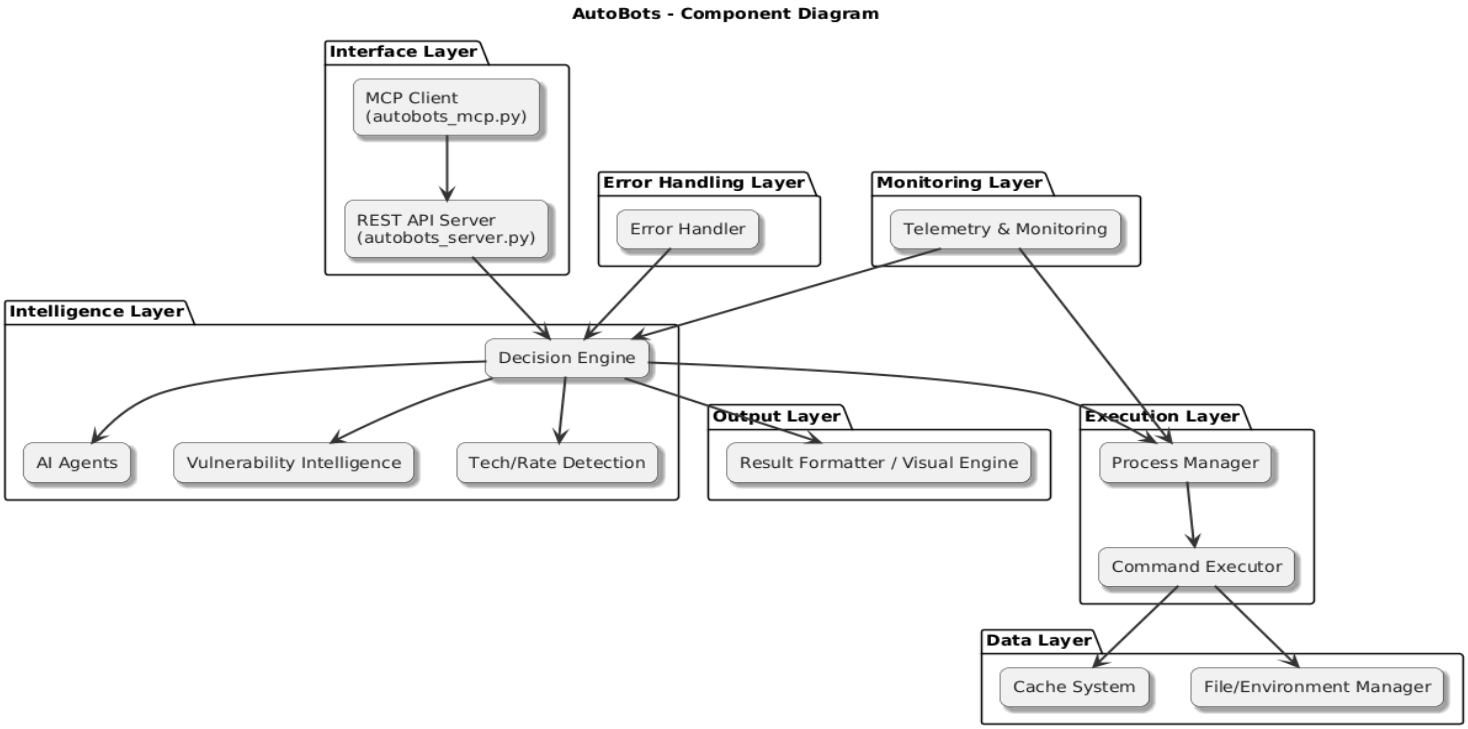

Figure 5: Components Diagram

5.3. Development View

Describes the internal software organization, including layered architecture, modules, code components, and their responsibilities during implementation.

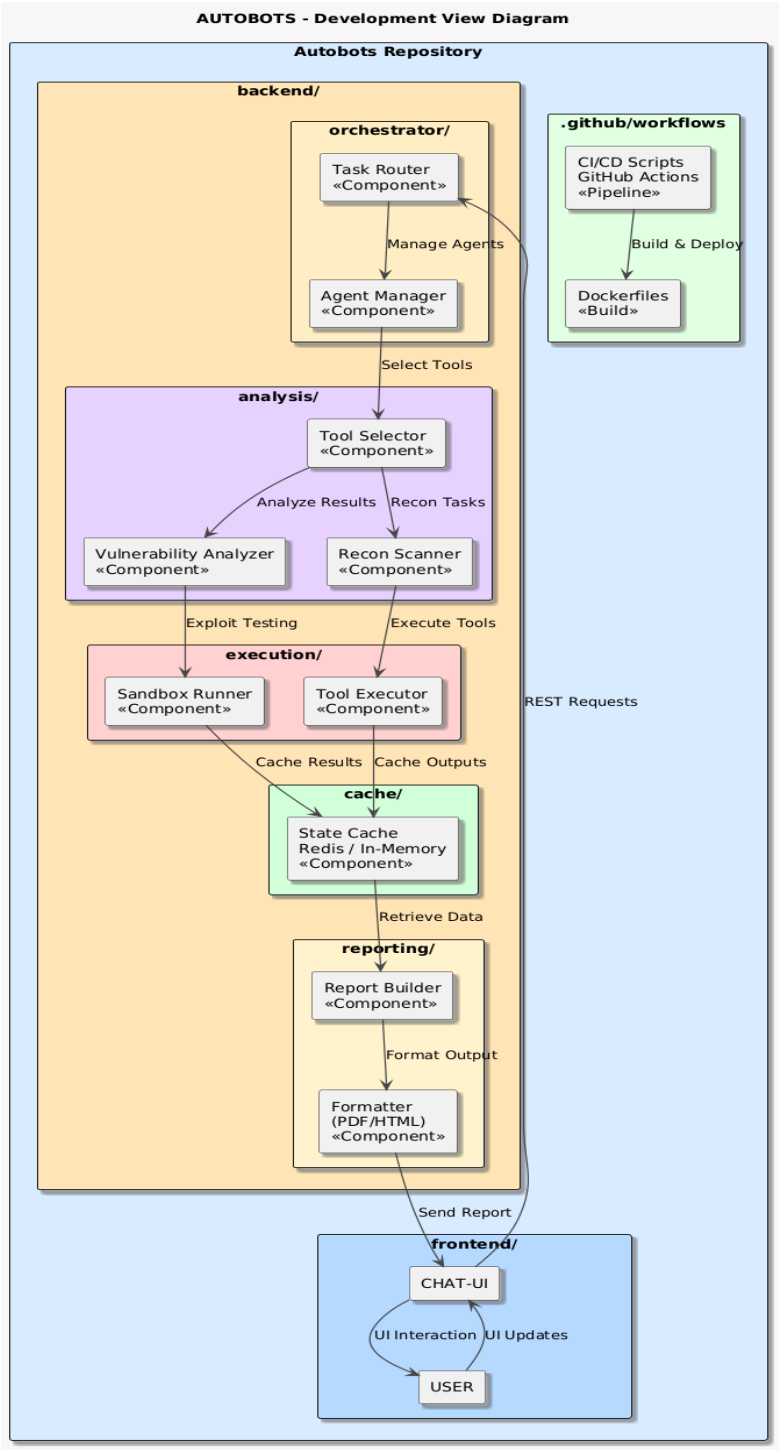

Figure 6: Development View

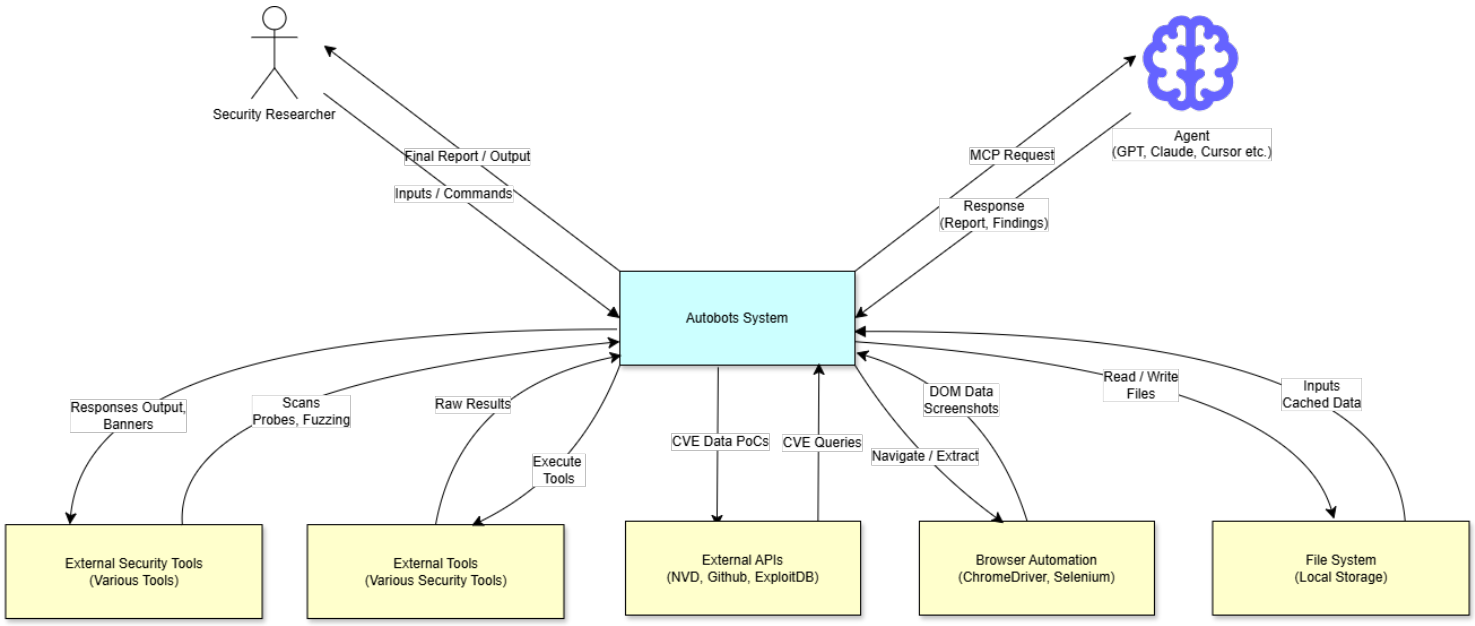

Figure 7: Context Diagram

5.4. Process View

Explains the runtime behavior of AUTOBOTs, detailing workflows, agent interactions, task orchestration, concurrency, and data flow during autonomous penetration testing.

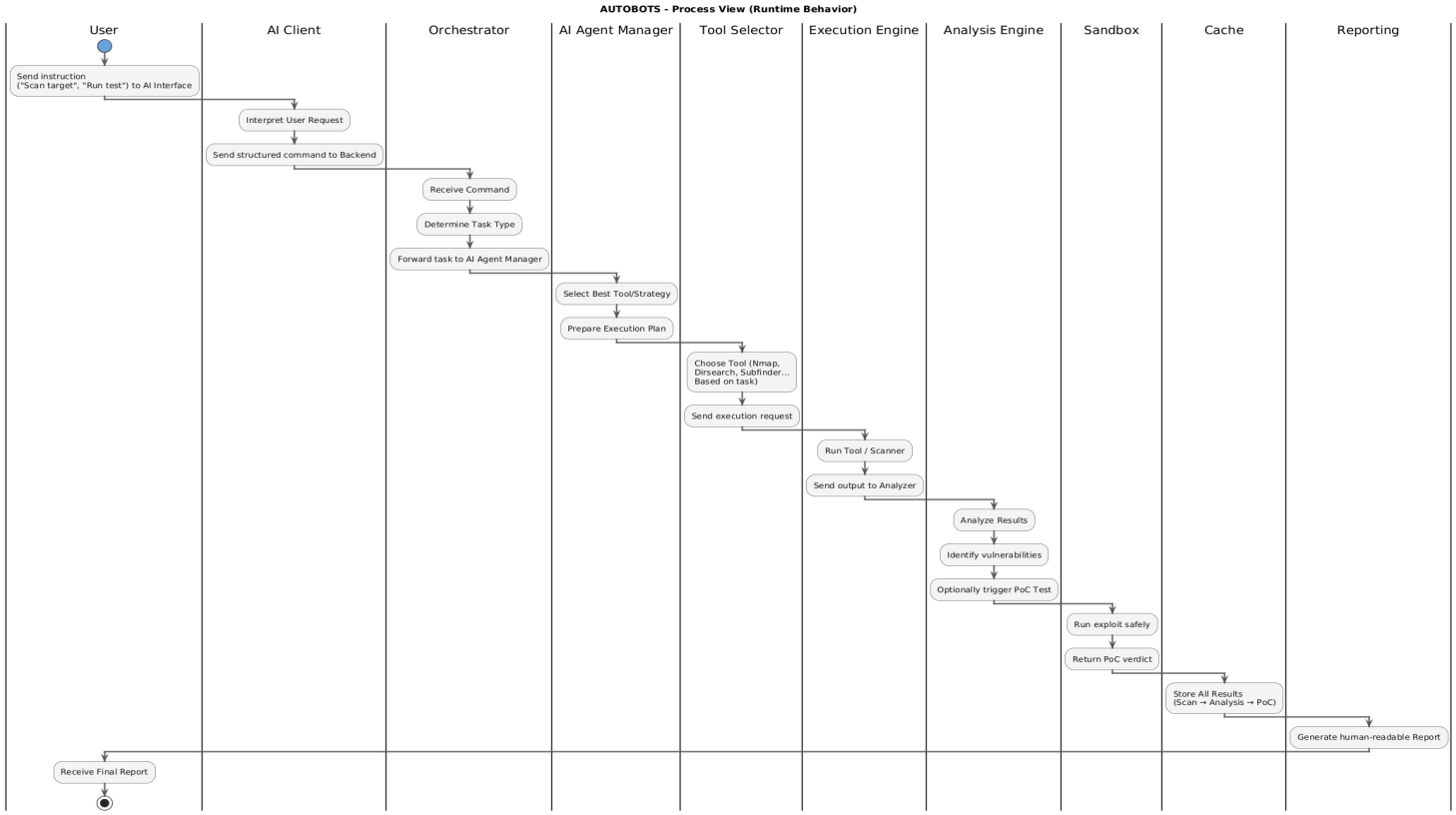

Figure 8: Process View

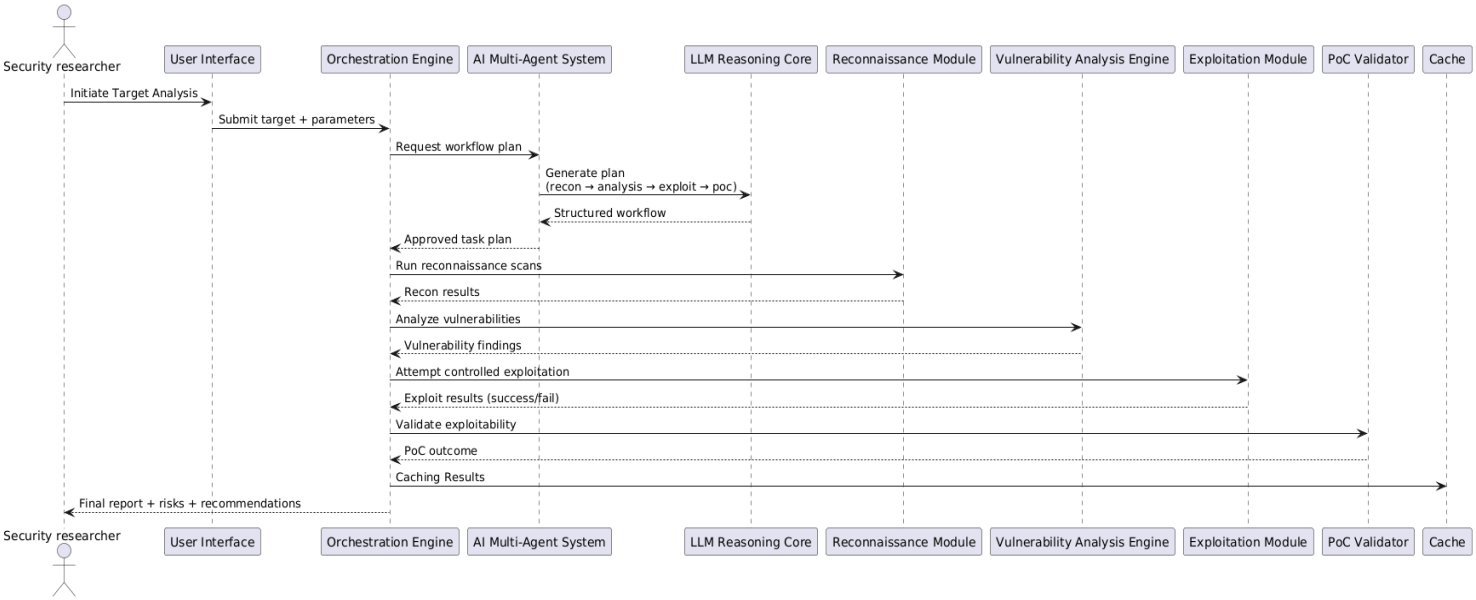

Figure 9: Sequence Diagram for a complete pen-testing process on a target

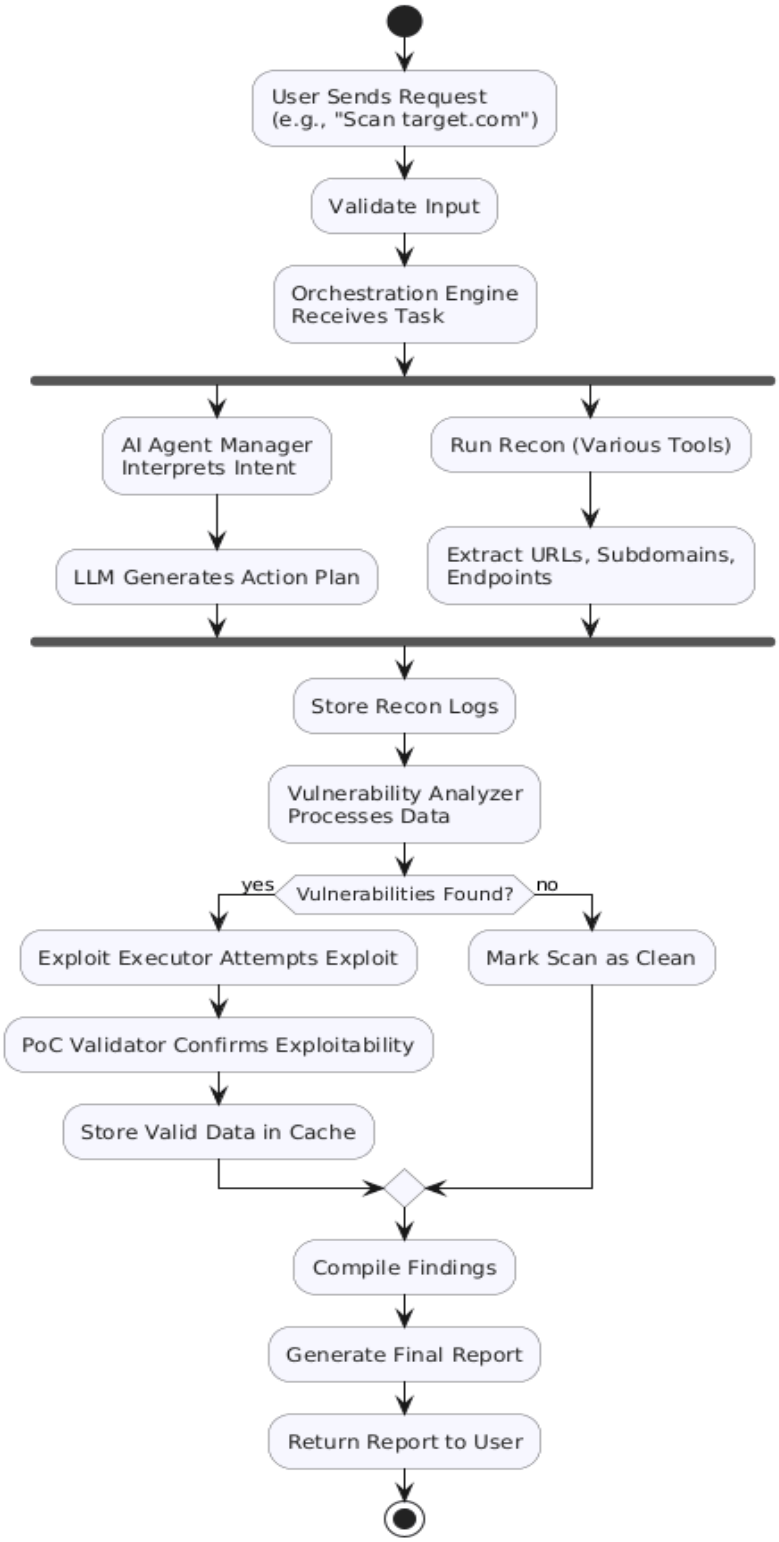

Figure 10: Activity Diagram

5.5. Physical View

Illustrates how AUTOBOTs' components are deployed across client and server environments in the two-tier architecture, including resource allocation and network communication.

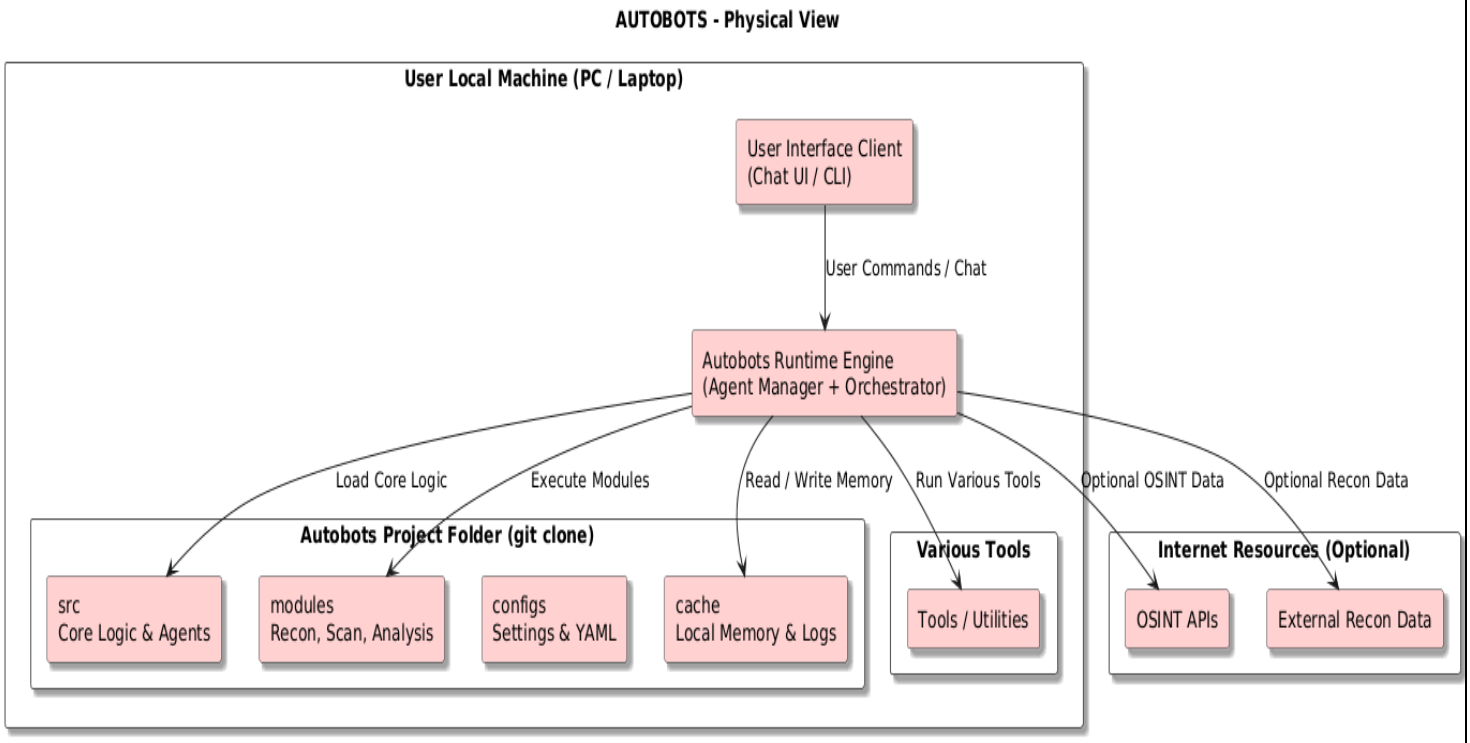

Figure 11: Physical View

5.6. User Interface Design

Figure 12: AUTOBOTs Local Application UI

Along with this, we also aim to introduce seamless integration in security researchers' day-to-day solutions e.g.:

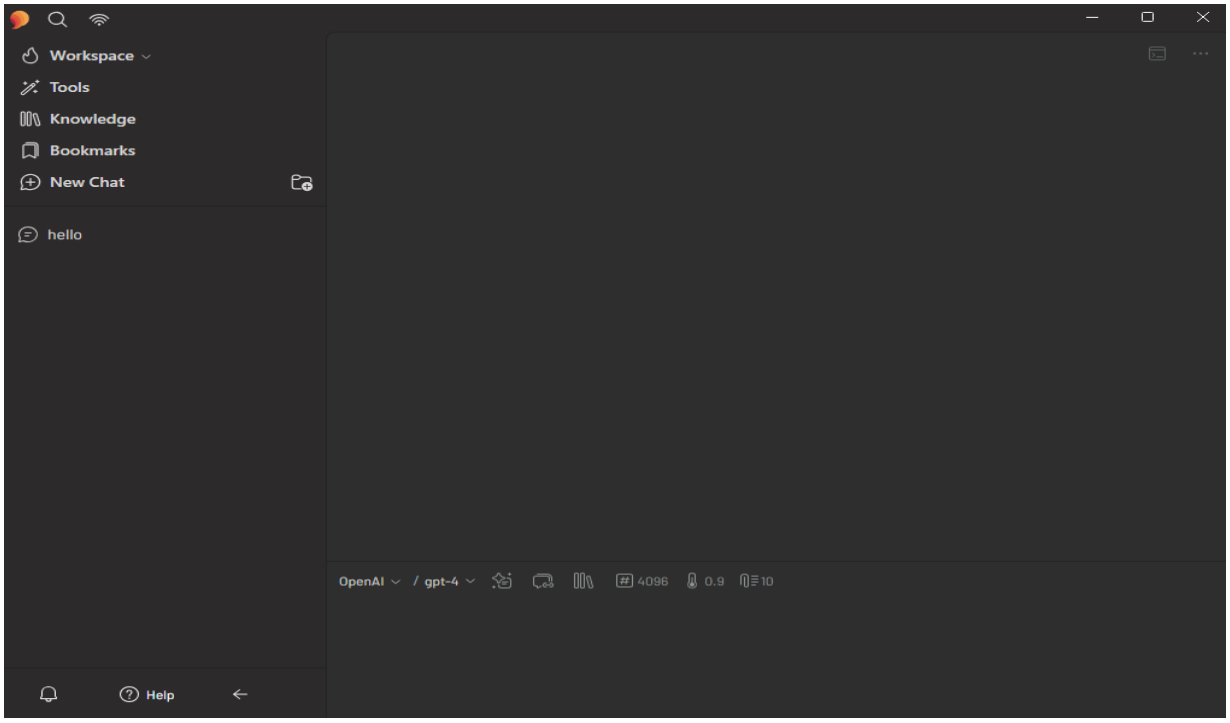

Figure 13: Integrated with 5IRE

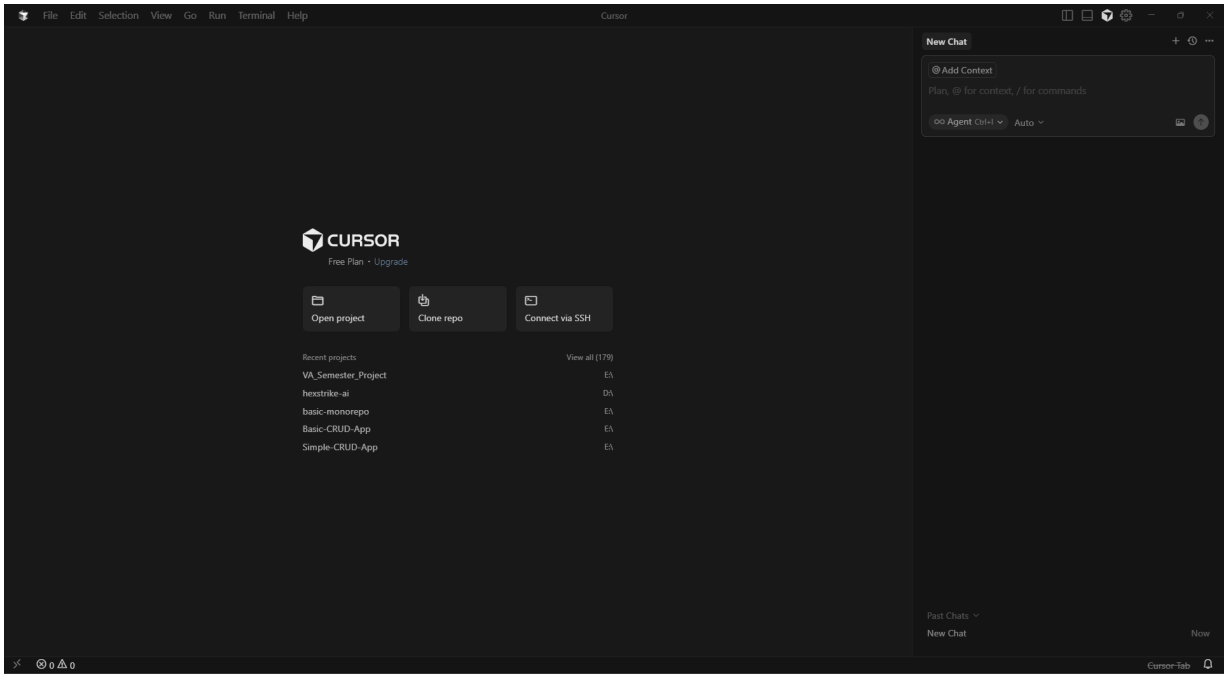

Figure 14: AUTOBOTs integrated with Cursor